

异世界冒险者公会管理系统

C++ 多线程编程大作业 难度：★★★★☆ | 满分：100 分

世界观设定

背景故事

在这片大陆上，冒险者公会是连接委托方与冒险者的核心机构。公会总部坐落于王都中心，每天处理来自四面八方的委托——讨伐魔物、探索遗迹、护送商队、采集药材，不一而足。

公会的日常运转依赖四个核心部门，它们相互协作，共同支撑起整个公会的业务：

[Phase 1] 派遣大厅 (Dispatch Hall)

大厅正面是一块巨大的任务告示板，委托方将任务写在羊皮纸上张贴上去。大厅设有多个服务窗口，每个窗口的工作人员不断从告示板取下任务，为冒险者办理接单手续。

问题在于：告示板是共享的，多个窗口同时操作时必须协调——不能两个窗口取走同一张任务，也不能在告示板空了的时候让窗口人员一直盯着（浪费精力），而应该等有新任务贴上来时再去取。

对应实现：QuestBoard<T>（线程安全队列）+ DispatchPool（线程池）

核心问题：多生产者多消费者的同步，以及“队列空时如何高效等待”

[Phase 2] 冒险者档案馆 (Guild Registry)

档案馆存放着所有注册冒险者的个人档案：战斗力、职业、称号、技能数据……每位冒险者的属性类型各不相同。档案馆按冒险者 ID 的字符串哈希分成 16 个区域（分片），A 区的查阅不影响 B 区的修改。

档案馆的访问规则是：**多人可以同时查阅同一区域，但修改时必须独占该区域**。此外，每张委托任务都有截止时间，后台有一只守护精灵定期巡逻，清理过期的委托档案；另一只精灵则盯着统计水晶球，定期汇报公会的运营数据。

对应实现：GuildRegistry（分片 KV 存储）+ ExpirySpirit / StatsDaemon（后台线程）

核心问题：读多写少场景下的读写锁，以及后台线程如何被“立即唤醒停止”而不是等到下次巡逻

[Phase 3] 装备交易所 (Trade System)

公会开设了装备交易所，冒险者之间可以互换装备。交易必须是原子的——不能出现“勇者 A 把圣剑交出去了，但魔法师 B 的法杖还没到手”的情况，否则公会将面临大量纠纷。

更棘手的是死锁问题：假设 A 想用圣剑换 B 的法杖，同时 B 也想用法杖换 A 的圣剑——A 锁住圣剑等法杖，B 锁住法杖等圣剑，两人永远僵持。公会的解决方案是：**所有交易必须按装备名称的字典序依次锁定**，从根本上杜绝循环等待。

对应实现：Transaction（事务 + write-set + commit/rollback）

核心问题：多把锁的死锁避免，以及事务内“读自己的未提交写入”（read-your-writes）

[Phase 4] 远征系统 (Expedition System)

远征队在外执行任务时，经常需要远程查阅档案馆的资料——查询冒险者属性、注册新成员、注销离队者。这些请求通过“通信水晶”（std::future）发出后，远征队不必干等回复，可以继续行军、扎营、侦察，等真正需要结果时再查看水晶。公会也可以同时派出多支远征队（BatchDispatcher），所有队伍并行执行各自的任務。

对应实现：ExpeditionClient（异步接口）+ BatchDispatcher（批量并行执行）

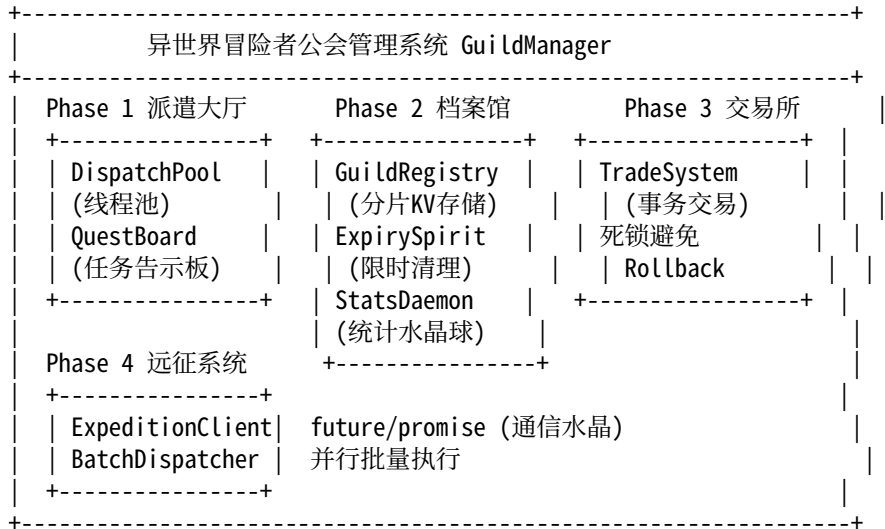
核心问题：如何把同步的档案馆操作包装成“提交后立即返回”的异步接口，以及如何并行等待多个任务全部完成

你的任务

作为技术总监，你需要为上述四个部门分别实现对应的系统模块。每个模块的接口（头文件/除了模板类）已经设计好，你只需要填写 `src/` 目录下各 `.cpp` 文件中标注 `TODO` 的部分。

所有模块都配有自动化测试，实现正确后测试应全部通过。

系统架构



核心机制详解

`std::thread` — 公会工作人员

`std::thread` 代表一个独立执行的线程，就像公会雇佣的一名工作人员。

```
// 创建一个线程，执行 lambda
std::thread worker([] {
    std::cout << " 我在独立线程里工作! \n";
});
worker.join(); // 等待该线程结束，否则析构时程序崩溃
```

关键规则：- `std::thread` 不可拷贝，只能移动 - 析构前必须 `join()` (等待) 或 `detach()` (放弃管理)，否则调用 `std::terminate()`
- 线程函数抛出的异常不会传播到创建线程，必须用 `future` 捕获

`std::mutex` + `std::lock_guard` — 告示板的独占锁

多个线程同时修改同一数据会产生数据竞争（data race），结果未定义。`std::mutex` 保证同一时刻只有一个线程进入临界区。

```
std::mutex mtx;
int counter = 0;

// 错误：多线程同时 ++counter，结果不确定
// 正确：
void increment() {
    std::lock_guard<std::mutex> lock(mtx); // 构造时加锁
    ++counter;
} // lock 析构时自动解锁 (RAII)
```

`std::lock_guard` 是最简单的锁管理器，构造加锁、析构解锁，**不能手动解锁**，适合简单场景。

`std::unique_lock` + `std::condition_variable` — 告示板的等待/唤醒

当队列为空时，消费者线程不应该忙等（浪费 CPU），而应该**睡眠等待**，直到生产者放入数据后被唤醒。

```
std::mutex mtx;
std::condition_variable cv;
std::queue<int> queue;

// 消费者：等待直到队列非空
void consumer() {
    std::unique_lock<std::mutex> lock(mtx);
    cv.wait(lock, [] { return !queue.empty(); });
    // wait 内部：① 释放锁并睡眠 ② 被唤醒后重新加锁 ③ 检查条件
    // 若条件为 false（虚假唤醒），继续睡眠
    int val = queue.front(); queue.pop();
}

// 生产者：放入数据后唤醒消费者
void producer(int val) {
    {
        std::lock_guard<std::mutex> lock(mtx);
        queue.push(val);
    }
    cv.notify_one(); // 唤醒一个等待的消费者
}
```

`std::unique_lock` 比 `lock_guard` 更灵活：可以手动 `lock()/unlock()`，**条件变量必须配合 `unique_lock` 使用**。

虚假唤醒（spurious wakeup）：线程可能在没有 `notify` 的情况下被唤醒，因此 `wait` 必须传入判断条件的 `predicate`，内部会循环检查。

`std::packaged_task` + `std::future` — 任务通知书

线程池提交任务后，调用者需要一种方式获取任务的返回值。`std::packaged_task` 将可调用对象包装为可获取返回值的任务，`std::future` 是取回结果的“通知书”。

```
// 包装一个计算任务
std::packaged_task<int(int, int)> task([](int a, int b) { return a + b; });
std::future<int> future = task.get_future(); // 先拿到通知书

std::thread t(std::move(task), 3, 4); // packaged_task 只能移动！
t.join();

int result = future.get(); // 阻塞直到任务完成，返回 7
```

为什么 `packaged_task` 不能直接放入 `std::function`？ `std::function` 要求内部对象可拷贝，但 `packaged_task` 只能移动（含有 `promise`，不可拷贝）。思考：如何让一个不可拷贝的对象能被 `std::function` 捕获？

`std::shared_mutex` + `std::shared_lock` — 档案馆的读写锁

档案馆的访问模式是**读多写少**：大量冒险者同时查阅档案（读），偶尔才有人修改（写）。普通 `mutex` 让读者也互相阻塞，浪费并发性。`std::shared_mutex` 允许多个读者同时持有锁。

```
std::shared_mutex rw_mutex;
std::unordered_map<std::string, int> data;
```

```
// 读操作：共享锁，多线程可同时持有
int read(const std::string& key) {
    std::shared_lock lock(rw_mutex); // 共享读锁
    return data.at(key);
}
```

```
// 写操作：独占锁，同一时刻只有一个线程
void write(const std::string& key, int val) {
    std::unique_lock lock(rw_mutex); // 独占写锁
    data[key] = val;
}
```

规则： - 多个 `shared_lock` 可以同时存在（并发读） - `unique_lock` 存在时，所有其他锁（包括 `shared_lock`）都必须等待 - 读多写少时，读写锁的吞吐量远高于普通 `mutex`

哈希分片策略 — 降低锁竞争

当并发量很高时，即使是读写锁也会成为瓶颈——所有写操作仍然互相阻塞。**分片（Sharding）** 将数据分散到多个独立的桶中，每个桶有自己的锁，从而将锁竞争降低为原来的 $1/N$ 。

```
// 通用分片路由：将 key 映射到某个桶
// std::hash<T> 是标准库提供的哈希函数对象，可将任意可哈希类型映射为 size_t
size_t bucket_index = std::hash<std::string>{}(key) % num_buckets;
```

设计权衡： - 分片数越多，锁竞争越低，但内存开销越大 - 分片数通常选择质数或 2 的幂次，以减少哈希碰撞 - 跨分片操作（如事务涉及多个 key）需要同时持有多个锁，需注意死锁

`std::variant` + `std::visit` — 冒险者的多类型属性

冒险者的属性可以是整数（战斗力）、浮点数（幸运值）、字符串（称号）等不同类型。`std::variant` 是类型安全的联合体，在编译期确定可能的类型集合。

```
using Attribute = std::variant<int64_t, double, std::string>;
```

```
Attribute attr = int64_t(9999); // 存储整数
```

```
// std::visit: 对 variant 中的实际类型执行操作
std::visit([](auto&& val) {
    using T = std::decay_t<decltype(val)>;
    if constexpr (std::is_same_v<T, int64_t>) {
        std::cout << " 战斗力: " << val << "\n";
    } else if constexpr (std::is_same_v<T, std::string>) {
        std::cout << " 称号: " << val << "\n";
    }
}, attr);
```

```
// 直接取值（类型不匹配时抛出 std::bad_variant_access）
int64_t power = std::get<int64_t>(attr);
```

相比继承体系，`variant` 无需虚函数、无堆分配，性能更好，且类型集合在编译期固定。

`std::atomic` — 统计水晶球（无锁计数）

对于简单的计数器和标志位，用 `mutex` 加锁开销太大。`std::atomic` 利用 CPU 硬件指令保证操作的原子性，**无需加锁**。

```

std::atomic<uint64_t> query_count{0};
std::atomic<bool> stop_flag{false};

// 多线程安全地递增，无需 mutex
query_count.fetch_add(1);           // 原子加法，返回旧值
query_count++;                     // 等价写法

// 原子地读取
uint64_t current = query_count.load();

// CAS (Compare-And-Swap): 条件更新，用于无锁算法
uint64_t expected = 0;
query_count.compare_exchange_weak(expected, 100);
// 若当前值 == expected, 则设为 100 并返回 true; 否则将 expected 更新为当前值并返回 false

```

适用场景：计数器、标志位、简单状态机。复杂的多变量一致性仍需 mutex。

TTL 过期 + std::chrono — 任务限时机制

每个委托任务都有截止时间，超时未完成自动作废。std::chrono 提供类型安全的时间处理。

```
using namespace std::chrono_literals; // 启用 1s、500ms 等字面量
```

```

// 记录过期时间点
auto expire_at = std::chrono::steady_clock::now() + 3600s;

// 判断是否过期
bool is_expired() const {
    return std::chrono::steady_clock::now() > expire_at;
}

// 可中断的定时等待
std::condition_variable cv;
std::mutex mtx;
std::atomic<bool> stop{false};

std::unique_lock lock(mtx);
cv.wait_for(lock, 30s, [&] { return stop.load(); });
// 30s 后超时返回，或被 notify 提前唤醒

```

wait_for vs sleep_for：两者都能实现定时等待，但行为有重要区别。思考：守护精灵需要能“立即停止”，哪种方式更合适？为什么？

事务 + 死锁避免 — 装备交易所规则

什么是死锁？

线程A：锁住“圣剑” → 等待“法杖”
 线程B：锁住“法杖” → 等待“圣剑”
 → 两人互相等待，永远无法继续 → 死锁

死锁的四个必要条件（Coffman 条件）：互斥、持有并等待、不可抢占、循环等待。破坏任意一个即可避免死锁。最实用的方法是破坏“循环等待”：所有线程按相同顺序获取锁。

```

// 错误：不同线程加锁顺序不同，可能死锁
void transfer_A_to_B() { lock(sword); lock(staff); ... }
void transfer_B_to_A() { lock(staff); lock(sword); ... }

```

```
// 正确：所有线程按相同顺序获取锁（破坏循环等待条件）
// 思考：如何确定" 相同顺序"？
```

`std::lock(m1, m2, ...)` 可以同时获取多把锁，内部使用死锁避免算法（尝试-回退策略）。获取后用 `std::lock_guard(m, std::adopt_lock)` 接管，`adopt_lock` 告诉 `lock_guard` “锁已经锁上了，只管释放”。

事务的 ACID 特性：- A（原子性）：commit 要么全部成功，要么全部不发生 - C（一致性）：交易前后总金币数不变 - I（隔离性）：事务内能读到自己的未提交写入（read-your-writes） - D（持久性）：本作业为内存存储，不要求持久化

std::any — 交易附加信息（类型擦除）

`std::any` 可以存储任意可拷贝类型的值，类型在运行时确定。适合需要存储 “不知道是什么类型” 的场景。

```
std::any context;
```

```
context = std::string(" 购买圣剑"); // 存储 string
context = 42;                       // 覆盖为 int
```

```
// 安全取值（指针版本，类型不匹配返回 nullptr，不抛异常）
if (auto* p = std::any_cast<int>(&context)) {
    std::cout << " 交易编号: " << *p << "\n";
}
```

```
// 不安全取值（类型不匹配抛出 std::bad_any_cast）
std::string note = std::any_cast<std::string>(context);
```

与 `void*` 相比，`std::any` 是类型安全的，会在运行时检查类型；与 `variant` 相比，`any` 的类型集合是开放的，但失去了编译期检查。

std::future / **std::promise** — 远征通信水晶

`future` 和 `promise` 是一对配套工具，用于在线程间传递异步结果。

```
// promise 是" 写端", future 是" 读端"
std::promise<int> promise;
std::future<int> future = promise.get_future();
```

```
// 在另一个线程中设置结果
std::thread t([p = std::move(promise)]() mutable {
    std::this_thread::sleep_for(1s);
    p.set_value(42); // 设置结果，唤醒等待的 future
});
```

```
// 主线程等待结果
int result = future.get(); // 阻塞直到 promise.set_value() 被调用
t.join();
```

`std::async` 是更简便的方式，自动创建线程并返回 `future`：

```
auto future = std::async(std::launch::async, [] {
    return compute_something();
});
// 做其他事情...
```

```
auto result = future.get(); // 需要时再等待
```

future 的状态：- 有效（valid）：关联了 `promise` 或 `packaged_task` - 就绪（ready）：结果已设置，`get()` 立即返回 - 无效：默认构造或已调用过 `get()`

std::invoke — 统一调用接口

std::invoke 统一了所有可调用对象的调用方式：普通函数、lambda、成员函数指针、函数对象。

```
// 普通函数
int add(int a, int b) { return a + b; }
std::invoke(add, 1, 2); // → 3

// Lambda
auto mul = [](int a, int b) { return a * b; };
std::invoke(mul, 3, 4); // → 12

// 成员函数（需要传入对象）
struct Foo { int val; int get() const { return val; } };
Foo foo{42};
std::invoke(&Foo::get, foo); // → 42

// 配合 std::invoke_result_t 推导返回类型
template<typename F, typename... Args>
auto call(F&& f, Args&&... args) -> std::invoke_result_t<F, Args...> {
    return std::invoke(std::forward<F>(f), std::forward<Args>(args)...);
}
```

在线程池的 dispatch 模板函数中，std::invoke 保证能正确调用任意类型的可调用对象，std::invoke_result_t 可以在编译期推导其返回类型。

dispatch 的模板技巧 — 线程池实现的核心难点

dispatch 的签名看起来简单，但实现时有几个容易卡住的地方：

万能引用与完美转发

```
template<typename F, typename... Args>
auto dispatch(F &&f, Args &&... args) -> std::future<std::invoke_result_t<F, Args...>>;
```

这里的 F && 不是右值引用，而是**万能引用**（universal reference）——当 F 是模板参数时，&& 既能绑定左值也能绑定右值。传入 lambda 临时对象时绑定右值，传入函数对象变量时绑定左值。

在函数体内转发参数时，必须用 std::forward<F>(f) 而不是直接用 f，否则右值会退化为左值，导致不必要的拷贝甚至编译错误。

std::invoke_result_t 推导返回类型

```
using R = std::invoke_result_t<F, Args...>; // 编译期推导 f(args...) 的返回类型
```

std::invoke_result_t<F, Args...> 是 C++17 的特性，等价于 C++11 的 typename std::result_of<F(Args...)>::type。它在编译期计算出调用 f(args...) 的返回类型，让 dispatch 能返回正确的 std::future<R>。

参数绑定 用户调用 dispatch(f, arg1, arg2)，但 board_ 只接受 std::function<void()>（无参）。需要在创建 packaged_task 时把参数通过 lambda 捕获绑定进去，让 worker 线程调用时不需要传参。

四个阶段

Phase 1: 派遣大厅（25 分）

任务：实现线程安全的任务告示板和线程池。

需要实现的文件： - include/quest_board.h 中的 QuestBoard<T> 类（模板类，全在头文件） - src/dispatch_pool.cpp 中的 DispatchPool 类

关键知识点: - `std::mutex` + `std::lock_guard` (简单加锁) - `std::unique_lock` + `std::condition_variable` (等待/唤醒) - `std::packaged_task` + `std::future` (异步结果) - 模板 + 完美转发 + `std::invoke_result_t` - RAII (析构函数保证资源释放)

测试: `ctest -R test_quest_board` 和 `ctest -R test_dispatch_pool`

Phase 2: 冒险者档案馆 (30 分)

任务: 实现分片并发 KV 存储和后台守护精灵。

需要实现的文件: - `src/guild_registry.cpp` 中的 `Shard` 和 `GuildRegistry` 类 - `src/guild_daemon.cpp` 中的 `ExpirySpirit` 和 `StatsDaemon` 类

关键知识点: - `std::shared_mutex` + `std::shared_lock` (读写锁) - `std::variant` + `std::visit` (多类型值) - `std::atomic` (无锁计数器) - `std::chrono` + 时间字面量 (1s, 500ms) - 结构化绑定 `auto [iter, inserted] = ...` - `std::condition_variable::wait_for` (可中断的定时等待)

测试: `ctest -R test_guild_registry` 和 `ctest -R test_guild_daemon`

Phase 3: 装备交易所 (25 分)

任务: 实现支持死锁避免的事务系统。

需要实现的文件: - `src/trade_system.cpp` 中的 `Transaction` 类

关键知识点: - 死锁的产生原因和避免策略 (按序加锁) - `std::lock()` (同时获取多把锁) - `std::adopt_lock` (接管已锁定的锁) - `std::any` + `std::any_cast` (类型擦除) - 事务的 ACID 特性

核心挑战: `commit()` 方法中的死锁避免实现

测试: `ctest -R test_trade_system` (包含金币守恒测试)

Phase 4: 远征系统 (20 分)

任务: 实现异步客户端和批量派遣器。

需要实现的文件: - `src/expedition.cpp` 中的 `ExpeditionClient` 和 `BatchDispatcher` 类

核心思路:

`ExpeditionClient` 的本质是把同步的档案馆操作包装成异步接口。以 `async_query` 为例: - 同步版本: `registry_.query(key)` — 调用后阻塞直到返回结果 - 异步版本: 把查询操作提交给 `pool_.dispatch`, 立即拿到 `future`, 查询在 `worker` 线程里执行, 调用方继续做其他事, 需要结果时再 `.get()`

`BatchDispatcher` 的核心是并行: 对每个任务都调用 `pool_.dispatch` 提交, 收集所有 `future`, 最后统一 `.get()` 等待全部完成。提交是串行的, 但执行是并行的。

关键知识点: - `std::future` / `std::promise` (异步结果传递) - `std::invoke` (统一调用接口, 处理不同类型的任务) - `std::function` (类型擦除的可调用对象)

测试: `ctest -R test_expedition`

构建和测试

克隆/下载项目后:

```
mkdir build && cd build
```

普通构建

```
cmake .. -DCMAKE_BUILD_TYPE=Debug
```

```
make -j$(nproc)
```



```
# 运行所有测试
ctest --output-on-failure

# 运行特定阶段的测试
ctest -R test_quest_board
ctest -R test_dispatch_pool
ctest -R test_guild_registry
ctest -R test_guild_daemon
ctest -R test_trade_system
ctest -R test_expedition
ctest -R stress_test

# 开启 ThreadSanitizer (检测数据竞争)
cmake .. -DENABLE_TSAN=ON
make -j$(nproc)
ctest --output-on-failure

# 运行性能基准
./benchmark/benchmark_gui
```

评分标准

阶段	分值
Phase 1 派遣大厅	25
Phase 2 档案馆	30
Phase 3 交易所	25
Phase 4 远征	20

细项说明:

- **Phase 1:** 告示板功能 (5) + 线程安全 (3) + dispatch 返回 future(5) + 模板实现 (4) + RAII 关闭 (4) + 并发测试 (4)
- **Phase 2:** 基本 CRUD(6) + 读写锁 (6) + 分片 (4) + TTL 过期 (5) + 原子统计 (4) + variant 使用 (5)
- **Phase 3:** 基本事务流程 (8) + read-your-writes(6) + rollback(6) + any 上下文 (5)
- **Phase 4:** async 接口 (6) + future 正确 (5) + batch 执行 (5) + invoke 使用 (4)

加分项 (最多 +25 分, 均有自动测试):

加分项	分值	测试方式
Phase 4 实现带回调的异步接口	+5	ctest -L bonus -R BonusCallback
Phase 3 事务只锁涉及的 shard (细粒度锁)	+20	ctest -L bonus -R BonusShardLock

详细的加分项实现原理、步骤和常见陷阱, 见 docs/BONUS_GUIDE.md。

运行所有加分项测试

```
ctest -L bonus --output-on-failure
```

注意: ThreadSanitizer 是验证线程安全的工具, 建议用于自查, 但不计入评分。开启方式: `cmake .. -DENABLE_TSAN=ON -DCMAKE_BUILD_TYPE=Debug && make -j$(nproc) && ctest`

提交要求

1. 只提交 src/ 目录下的 .cpp 文件, include/ 目录下的 .h 文件 (不要添加新的函数定义, 不要修改 tests/)
2. 代码需要能通过 `cmake .. && make` 编译

祝各位冒险者编程顺利, 早日成为 S 级程序员!